

11 — Załącznik B: Diagramy

[← Powrót do README](#) (../README.md) | [← Słownik](#) (./glossary.md) | [Następna: Specyfikacja API](#) → (./api-specifications.md)

C4 Context Diagram (Poziom 1)

```
graph TB
    subgraph "Aktorzy"
        OP["👤 Security Operator"]
        AN["👤 Threat Analyst"]
    end

    subgraph "Systemy Konsumenckie"
        SIEM["📊 SIEM Systems<br/>Splunk, Sentinel, ArcSight"]
        FW["🔥 Firewalls<br/>FortiGate, Palo Alto"]
    end

    IOC["🛡️ IOC Service<br/>Threat Intelligence Aggregator"]

    subgraph "Źródła TI"
        CS["CrowdSec"]
        MISP_S["MISP"]
        MWDB_S["MWDB"]
        MB["MalwareBazaar"]
        AC["abuse.ch (5 svc)"]
    end

    OP -->|"HTTPS"| IOC
    AN -->|"HTTPS"| IOC
    SIEM -->|"REST API"| IOC
    FW -->|"HTTPS"| IOC

    IOC -.->|"HTTPS"| CS
    IOC -.->|"PyMISP"| MISP_S
    IOC -.->|"HTTPS"| MWDB_S
    IOC -.->|"HTTPS"| MB
    IOC -.->|"HTTPS"| AC

    style IOC fill:#4CAF50,color:#fff
```

C4 Container Diagram (Poziom 2)

```
graph TB
    subgraph "External"
        CLIENT["🌐 Clients"]
        FEEDS["🔊 Threat Feeds"]
    end

    subgraph "IOC Service"
        NGINX["Nginx<br/>:443<br/>TLS + WAF"]
        APP["Flask App<br/>:8080<br/>API + Web UI"]
        WORKER["Worker<br/>Background<br/>Feed Sync"]
        PG["PostgreSQL 16<br/>9 Tables<br/>JSONB + FTS"]
        REDIS["Redis 7<br/>Cache<br/>Rate Limiter"]
        PROM["Prometheus<br/>Metrics"]
        GRAF["Grafana<br/>Dashboards"]
    end

    CLIENT --> NGINX --> APP
    APP --> PG
    APP --> REDIS
    WORKER --> PG
    WORKER --> REDIS
    WORKER --> FEEDS
    APP --> PROM --> GRAF

    style APP fill:#4CAF50,color:#fff
    style WORKER fill:#FF9800,color:#fff
    style PG fill:#2196F3,color:#fff
    style REDIS fill:#F44336,color:#fff
```

C4 Component Diagram — Application (Poziom 3)

```

graph TB
    subgraph "HTTP Layer"
        AUTH["Auth Middleware"]
        API["API v1 Routes"]
        ADMIN["Admin Routes"]
        WEBUI["Web UI"]
    end

    subgraph "Service Layer"
        IND_SVC["IndicatorService"]
        FEED_SVC["FeedService"]
        EXPORT_SVC["ExportService"]
        AUDIT_SVC["AuditService"]
    end

    subgraph "Integration Layer"
        REGISTRY["AdapterRegistry"]
        PIPELINE["IngestionPipeline"]
        ADAPTERS["Feed Adapters<br/>(5+)"]
    end

    subgraph "Data Layer"
        MODELS["SQLAlchemy Models"]
        CACHE["Redis Cache"]
    end

    AUTH --> API
    AUTH --> ADMIN
    API --> IND_SVC
    API --> EXPORT_SVC
    ADMIN --> FEED_SVC
    ADMIN --> AUDIT_SVC

    FEED_SVC --> REGISTRY
    REGISTRY --> ADAPTERS
    ADAPTERS --> PIPELINE
    PIPELINE --> MODELS

    IND_SVC --> MODELS
    IND_SVC --> CACHE
    EXPORT_SVC --> CACHE

    style REGISTRY fill:#4488ff,color:#fff
    style PIPELINE fill:#4488ff,color:#fff
    style AUTH fill:#ff4444,color:#fff

```

Sequence Diagram — Feed Ingestion

```
sequenceDiagram
    participant Scheduler
    participant Registry
    participant Adapter
    participant Connector as HTTP Connector
    participant CircuitBreaker
    participant Pipeline
    participant DB as PostgreSQL
    participant Metrics

    Scheduler->>Registry: get("crowdsec")
    Registry-->>Scheduler: CrowdSecAdapter

    Scheduler->>CircuitBreaker: is_open("crowdsec")?
    CircuitBreaker-->>Scheduler: false (closed)

    Scheduler->>Adapter: fetch(config, context)
    Adapter->>Connector: request_json(url, headers)
    Connector->>Connector: throttle + retry
    Connector-->>Adapter: raw_data[]
    Adapter->>Adapter: _map_to_canonical(raw)
    Adapter-->>Scheduler: FetchResult(items, stats)

    Scheduler->>Pipeline: process(items, "crowdsec")
    Pipeline->>Pipeline: validate → canonicalize → dedup
    Pipeline->>DB: UPSERT indicators
    Pipeline->>DB: DEACTIVATE missing
    Pipeline->>DB: UPDATE feed_stats
    Pipeline-->>Scheduler: PipelineResult

    Scheduler->>CircuitBreaker: record_success("crowdsec")
    Scheduler->>Metrics: observe(duration, count)
```

Sequence Diagram — Dodawanie Nowego Adaptera

```
sequenceDiagram
    participant Dev as Developer
    participant Code as app/adapters/new.py
    participant Config as feeds.yaml
    participant Registry
    participant Tests
    participant CI

    Dev->>Code: Implement NewSourceAdapter
    Note over Code: source_id, capabilities,<br/>_fetch_raw, _map_to_canonical

    Dev->>Config: Add feed config section

    Dev->>Tests: Run contract tests
    Tests->>Registry: auto_discover()
    Registry->>Code: Register NewSourceAdapter
    Tests->>Tests: validate protocol compliance
    Tests-->>Dev:  All contract tests pass

    Dev->>CI: Push PR (~200 LOC)
    CI->>CI: lint  test  security  build
    CI-->>Dev:  CI green

    Note over Registry: Deploy: auto-discovery<br/>picks up new adapter
    Registry->>Registry: validate_config()
    Registry->>Registry: test_connection()
    Registry->>Registry: Schedule sync
```

Deployment Diagram

```
graph TB
    subgraph "Host Server"
        subgraph "Docke Network: frontend"
            NGINX["Nginx Container<br/>:443 → :8080"]
        end

        subgraph "Docke Network: backend"
            APP["App Container<br/>Gunicorn :8080<br/>2 CPU / 2GB RAM"]
            WORKER["Worker Container<br/>Scheduler Loop<br/>1 CPU / 1GB RAM"]
            PG["PostgreSQL Container<br/>:5432<br/>Volume: pg_data"]
            REDIS["Redis Container<br/>:6379<br/>Volume: redis_data"]
        end

        subgraph "Docke Network: monitoring"
            PROM["Prometheus<br/>:9090"]
            GRAF["Grafana<br/>:3000"]
        end
    end

    NGINX --> APP
    APP --> PG
    APP --> REDIS
    WORKER --> PG
    WORKER --> REDIS
    PROM --> APP
    PROM --> GRAF

    style NGINX fill:#9C27B0,color:#fff
    style APP fill:#4CAF50,color:#fff
    style WORKER fill:#FF9800,color:#fff
    style PG fill:#2196F3,color:#fff
    style REDIS fill:#F44336,color:#fff
```

Network Diagram

```
graph LR
    subgraph "Internet"
        EXTERNAL["🌐 External Clients<br/>SIEM, Firewalls, Browsers"]
        FEEDS["🔊 Threat Feeds<br/>CrowdSec, MISP, etc."]
    end

    subgraph "DMZ"
        FW1["🔥 Firewall"]
        NGINX["Nginx :443"]
    end

    subgraph "Application Zone"
        APP["Flask App :8080"]
        WORKER["Worker"]
    end

    subgraph "Data Zone"
        PG["PostgreSQL :5432"]
        REDIS["Redis :6379"]
    end

    EXTERNAL -->|"HTTPS :443"| FW1
    FW1 --> NGINX
    NGINX -->|"HTTP :8080"| APP
    WORKER -->|"HTTPS"| FEEDS

    APP --> PG
    APP --> REDIS
    WORKER --> PG
    WORKER --> REDIS

    style FW1 fill:#ff4444,color:#fff
```